

Beyond Boolean Intersections: A Verified Bisimulation and Trace Decompiler for Causal Graphs

LUCIA YIZI ZHANG, Shanghai Lixin University of Accounting and Finance, China

We mechanize a verified bisimulation between two standard characterizations of d-separation in Lean 4, expose a concrete counterexample to unrestricted equivalence, and build a constructive decompiler that synthesizes exploitability-certificate witnesses from graph reachability alone.

The two characterizations—trail blocking [8] and moralized ancestral-graph separation [5]—have been treated as unconditionally equivalent in the literature. We machine-check a counterexample showing that the equivalence fails when an endpoint node lies in the conditioning set Z , identify the minimal repair ($\text{DisjointSets } X \ Y \ Z$, an affine ownership constraint), and discharge both directions of the corrected equivalence with no proof debt.

The forward direction is a *certified trace optimizer*: it compiles any active trail through a Bayes-ball state machine into a compressed moral-graph walk, certifying soundness under the exact domain restriction that repairs equivalence. The reverse direction is a *constructive decompiler of semantic flow*: from moral-graph reachability we reconstruct an explicit active-trail witness, including a proved bad-collider normalization ($\text{route_improves_of_bad}$) that strictly decreases a well-founded measure. Both directions compose into a closed equivalence on the pairwise-disjoint domain ($\text{dSeparated_iff_dSeparates}$); the graph core carries no proof debt.

The verified bisimulation serves as a zero-sorry graph-semantic foundation for certified Shannon-style leakage bounds. The local `InfoTheoryBridge.lean` file contains two intentional scaffold stubs restating results from a separate anonymized artifact [1]; two named technical debts in the quantitative chain (the Verma–Pearl factorization derivation and the dual KL capacity sufficiency theorem) are identified in Section 8 but do not affect the graph core.

1 Introduction

Formalizing both standard characterizations of d-separation in Lean 4’s dependent type system yields a mechanized counterexample: the unrestricted equivalence between Pearl’s trail-blocking predicate and Lauritzen’s moralized ancestral-graph separation is *false*. When an endpoint node lies in the conditioning set Z , the moral-graph construction deletes that endpoint while the trail predicate still admits the one-edge trail that has no internal triple.

The counterexample lives on the simplest possible directed acyclic graph: a chain of three nodes $0 \rightarrow 1 \rightarrow 2$. We show in Section 4 that setting $X = \{0\}$, $Y = \{1\}$, and $Z = \{0\}$ breaks equivalence: moral-graph separation unconditionally deletes Z (disconnecting X from Y), while trail blocking leaves the one-edge trail $0 \rightarrow 1$ unblocked.

We mechanize this discrepancy and its repair, treating the two characterizations not as interchangeable definitions but as a *verified bisimulation* between two semantic layers:

- an **operational semantics** of active trails (the “execution” layer of information flow);
- a **static analysis** of moralized-graph reachability (the “compiled” layer).

The forward direction (soundness) is a *certified trace optimizer* that compiles an operational trace into static reachability evidence. The reverse direction (completeness) is a *constructive decompiler of semantic flow*: it reconstructs a concrete execution witness—an exploitability certificate for the information leak—from static reachability alone. The counterexample says exactly that the two languages are *not* bisimilar without a well-typed domain restriction.

1.1 Contributions

Contributions:

- (1) We machine-check a concrete counterexample (`dsep_complete_endpoint_in_Z_counterexample`) showing that unrestricted equivalence between trail blocking and moral-graph separation is false, identify the minimal repair (`DisjointSets X Y Z` as an affine ownership constraint), and prove that no counterexample survives the repair.
- (2) We formalize both d-separation characterizations in Lean 4 as a three-layer stack: type-safe graph syntax, certified intermediate representations (`MAGWalk`, `StaticRoute`, `OpenTrace`, `ActiveRoute`), and witness-extraction pipelines.
- (3) We mechanize the **certified trace optimizer** (soundness): an explicit compilation pipeline `Trail` $\rightarrow$ `BayesBallPath` $\rightarrow$ `MAGWalk` $\rightarrow$ `Reachable`, certified by node-survival proofs.
- (4) We mechanize the **constructive decompiler of semantic flow** (witness-producing completeness): a decompilation pipeline `Reachable` $\rightarrow$ `StaticRoute` $\rightarrow$ `OpenTrace` $\rightarrow$ `ActiveRoute` $\rightarrow \exists \text{ Trail}, \neg \text{isBlocked}$. Every stage is type-indexed and constructive, including a proved bad-collider normalization (`exists_split`, `ancestor_escape`, `route_improves_of_bad`) that strictly decreases a well-founded measure. The reverse pipeline carries no [sorry](#).
- (5) We close the **full equivalence** (`dSeparated_iff_dSeparates`): on the pairwise-disjoint domain, moral-graph separation holds if and only if every trail is blocked, with both directions discharged by the optimizer and the decompiler.
- (6) We bridge the graph core to a certified Shannon leakage bound via a companion anonymized artifact [1]. The explicit scaffold `InfoTheoryBridge.lean` restates the companion-proved capacity bounds. The remaining technical debts bounding this claim are localized to the quantitative pipeline and do not affect the graph-semantic core (discussed in Section 8).

The graph results are available as a standalone Lean 4 package. The decompiler produces not a mere existence proof but a computational witness—a Σ -type containing the actual trail and the proof that it is unblocked.

2 A Three-Layer Architecture

Our formal development uses three layers mirroring a verified compiler’s architecture.

2.1 Layer 1: Type-Safe Graph Syntax

Nodes are natural numbers, edges are a finite set of ordered pairs ($\text{Finset } (\mathbb{N} \times \mathbb{N})$), and acyclicity is well-foundedness of the edge relation. The key safety invariant is `DisjointSets X Y Z`, which requires pairwise disjointness of the three node sets. In the language of substructural types, this is an *affine ownership constraint*: the conditioned variables Z are borrowed by the query and cannot alias the source X or sink Y . As planned future work for the POPL track—not part of this artifact—we target a first-order surface calculus with an explicit environment and no higher-order binders, so the metatheory would stay focused on queries rather than substitution-heavy closure machinery. In that planned layer, surface graphs would carry a rank certificate for acyclicity, checked locally and then elaborated into intrinsic DAGs via `DAG.ofRank`.

Listing 1. DAG structure and safety predicate

```
structure DAG where
  nodes : Finset \N
  edges : Finset (\N \times \N)
  edges_subset : \forall {u v}, (u,v) \in edges -> u \in nodes \wedge v \in nodes
  acyclic : WellFounded fun u v => (u,v) \in edges
```

```
def DisjointSets (X Y Z : Finset \N) : Prop :=
  Disjoint X Y \wedge Disjoint X Z \wedge Disjoint Y Z
```

2.2 Layer 2: Certified Intermediate Representations

Between operational trails and graph reachability sits a family of IRs that preserve just enough structure for certified translation.

MAGWalk. A compressed walk language equivalent to moral-graph reachability. It has three constructors: reflexivity, a single directed or reversed edge (`MAGWalk.single`), and a “moral jump” over an active collider (`MAGWalk.jump`). The collider node is skipped, which is why compression is correct.

StaticRoute. A type-valued decompiler IR that keeps the witness hidden by graph reachability. Each step is either a forward edge, a backward edge, or a moral jump storing the common child:

Listing 2. Static route IR (excerpt)

```
inductive StaticStep (G : DAG) (X Y Z : Finset \N) : \N -> \N -> Type where
| directForward (hEdge : G.HasEdge u v) ...
| directBackward (hEdge : G.HasEdge v u) ...
| moralJump (huw : G.HasEdge u w) (hvw : G.HasEdge v w)
  (hw : w \in G.ancestralSubgraphNodes (X \cup Y \cup Z)) ...
```

OpenTrace. The compiler target before converting to an active route. Each cons node stores its oriented DAG traversal *and* a local non-blocking proof, making *OpenTrace* a type-valued certificate that every junction is Z-open.

2.3 Layer 3: Witness-Extraction Pipelines

The two pipelines are summarized in Figure 1.

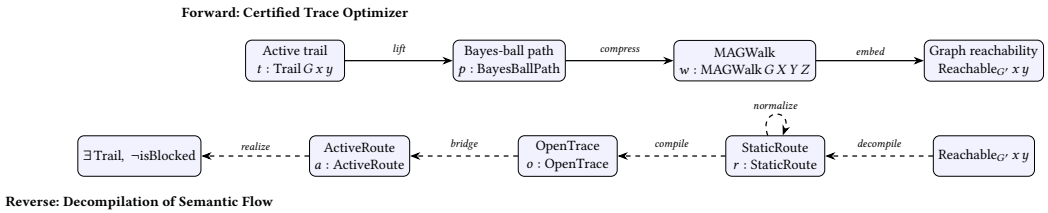


Fig. 1. The two witness-extraction pipelines. Solid arrows: forward (soundness). Dashed arrows: reverse (completeness). Every arrow, including the *normalize* step, is a total constructive function in Lean; the *normalize* step is a proved bad-collider minimization (Section 6).

3 Background: The Two Criteria

3.1 Trail Blocking (Operational Semantics)

An undirected *trail* from x to y in a DAG G is a finite walk that may follow edges forward or backward. A consecutive triple $a \rightarrow b \rightarrow c$ or $a \leftarrow b \rightarrow c$ is a *non-collider*; $a \rightarrow b \leftarrow c$ is a *collider*. A triple is *blocked* by Z if:

- it is a non-collider whose middle node is in Z ; or
- it is a collider whose middle node and all descendants are disjoint from Z .

A trail is *blocked* by Z if some internal triple is blocked. Then Z *d-separates* X from Y in the trail sense if every trail from X to Y is blocked by Z .

3.2 Moral Graph Separation (Static Analysis)

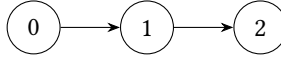
The *ancestral subgraph* of S is the subgraph induced by all ancestors of nodes in S . Its *moral graph* is obtained by (1) forgetting edge directions and (2) connecting all co-parents (pairs of nodes with a common child). The *d-separation graph* is the moral graph of the ancestral subgraph of $X \cup Y \cup Z$ with Z removed. Then Z d-separates X from Y in the graph sense if no node of X is connected to any node of Y in this graph.

3.3 Textbook Equivalence and Its Caveat

Standard references state that the two criteria are equivalent under the implicit assumption that X , Y , and Z are pairwise disjoint. In applications, however, the disjointness precondition is often omitted. Our counterexample (Section 4) shows that this omission is not benign.

4 The Bug in the Math Textbook

Our counterexample lives on the simplest non-trivial DAG:



Let $X = \{0\}$, $Y = \{1\}$, $Z = \{0\}$.

THEOREM 4.1 (ENDPOINT-IN- Z COUNTEREXAMPLE). *There exists a DAG G and finite sets $X, Y, Z \subseteq \mathbb{N}$ such that G d-separates X from Y in the moral-graph sense, but Z does not d-separate X from Y in the trail-blocking sense.*

Listing 3. Mechanized counterexample

```

theorem dsep_complete_endpoint_in_Z_counterexample :
  \exists (G : DAG) (X Y Z : Finset \N),
    DAG.dSeparated G X Y Z \wedge \neg dSeparates G X Y Z := by
  refine <chain3, {0}, {1}, {0}, ?_, ?_>
  ...

```

Why the disagreement happens. Trail blocking only inspects *internal* triples. The one-edge trail $0 \rightarrow 1$ has no internal triple at all, so nothing can block it. By contrast, the moral-graph construction deletes Z unconditionally; since $0 \in Z$, the edge $(0, 1)$ disappears from the moralized graph, and 0 and 1 are disconnected.

The standard repair. If we require $X \cap Z = \emptyset$ and $Y \cap Z = \emptyset$, the start node of any trail is never deleted, and the endpoint caveat disappears.

5 The Certified Trace Optimizer

Under $\text{DisjointSets } X \ Y \ Z$, we mechanize the soundness direction:

$$\text{DAG.dSeparated}(G, X, Y, Z) \implies \text{dSeparates}(G, X, Y, Z).$$

The proof follows a three-stage compilation pipeline.

Stage 1: Trail $\rightarrow$ Bayes-ball path. If a trail t from $x \in X$ to $y \in Y$ is not blocked by Z , and $x \notin Z$ (guaranteed by Disjoint $X Z$), then we lift t to an explicit Bayes-ball state path. The construction is by induction on the trail, analyzing each directional triple to show that the Bayes-ball state machine can traverse it.

Listing 4. Trail to Bayes-ball path

```
def bayesBallPath_of_active_trail_outOf
  (t : Trail G u v) (h_active : \neg t.isBlocked Z)
  (huZ : u \notin Z) :
  \Sigma final_dir, BayesBallPath G Z (u, outOf) (v, final_dir)
```

Stage 2: Certified membership. The Bayes-ball path contains intermediate states that must “survive” deletion of Z . We prove a certified version that guarantees every RequiredState node belongs to the d-separation graph nodes:

Listing 5. Certified Bayes-ball path

```
def bayesBallPathCert_of_active_trail_outOf
  (t : Trail G u v) (h_active : \neg t.isBlocked Z)
  (huZ : u \notin Z)
  (huA : u \in G.ancestralSubgraphNodes (X \cup Y \cup Z))
  (hvD : v \in G.dSeparationGraphNodes X Y Z) :
  \Sigma final_dir, {p : BayesBallPath ... //
    \forall {q}, BayesBallPath.RequiredState p q ->
      q.1 \in G.dSeparationGraphNodes X Y Z}
```

Stage 3: Compress to MAGWalk. A Bayes-ball path is a sequence of single-edge steps. The compress algorithm scans it with a two-step window: ordinary non-collider windows become a MAGWalk .single, while active-collider windows $(a, _) \rightarrow (b, \text{into}) \rightarrow (c, \text{outOf})$ become a single MAGWalk .jump over the collider b . The collider itself need not survive deletion of Z , which is why this compression is correct.

THEOREM 5.1 (SOUNDNESS). *For any DAG G and pairwise-disjoint finite sets $X, Y, Z \subseteq \mathbb{N}$, if G d-separates X from Y in the moral-graph sense, then every trail from X to Y is blocked by Z .*

Listing 6. Top-level soundness theorem

```
theorem dSeparated_of_dSeparated_disjoint
  (hXYZ : DisjointSets X Y Z)
  (hsep : DAG.dSeparated G X Y Z) : dSeparates G X Y Z
```

6 Decompilation of Semantic Flow

The reverse direction is the decompiler: from $\neg \text{DAG.dSeparated}$ it constructively reconstructs an active-trail witness—an exploitability certificate showing that the conditioning set Z fails to block the flow. The pipeline is:

$$\begin{aligned} \text{Reachable} &\rightarrow \text{StaticRoute} \rightarrow \text{NormalizedStaticRoute} \\ &\rightarrow \text{OpenTrace} \rightarrow \text{ActiveRoute} \rightarrow \exists \text{ Trail, } \neg \text{isBlocked.} \end{aligned}$$

6.1 From Reachability to StaticRoute

Graph reachability is a **Prop**-valued relation that hides the structure of each step. We introduce `StaticStep` and `StaticRoute` as type-valued evidence: a `StaticRoute` is an explicit list of directed edges, reversed edges, and moral jumps, each carrying the witness needed for later stages.

Listing 7. Reachability to static route

```
theorem nonemptyStaticRoute_of_dSeparationGraph_reachable
  (h : (G.dSeparationGraph X Y Z).Reachable u v) :
  Nonempty (StaticRoute G X Y Z u v)
```

6.2 Normalization: Eliminating Bad Colliders

A “bad collider” is a collider window whose middle node and all descendants are disjoint from Z . Such a collider is blocked, so it must be eliminated. Normalization may change endpoints: if a bad collider leaks ancestry into X or Y , we reroute the prefix or suffix to a different node in that set.

We package static routes as endpoint-carrying witnesses (`StaticRouteWitness`) and define a bad-collider count `routeBadCount`. The minimization is a `Nat.find` wrapper: if every nonzero-count witness can be strictly improved, a zero-count witness exists.

Listing 8. Bad-collider minimization wrapper

```
theorem normalized_route_exists_of_improves
  (hwit : \exists w : StaticRouteWitness G X Y Z, True)
  (himprove : \forall w, routeBadCount w \neq 0 ->
    \exists w', routeBadCount w' < routeBadCount w) :
  \exists w, routeBadCount w = 0
```

The improvement obligation is discharged by the core normalization theorem, which is fully proved (no **sorry**):

Listing 9. Strict improvement for non-normalized routes

```
theorem route_improves_of_bad
  (w : StaticRouteWitness G X Y Z) (hbad : routeBadCount w \neq 0) :
  \exists w' : StaticRouteWitness G X Y Z,
  routeBadCount w' < routeBadCount w
```

The proof is a short dispatcher over three established lemmas. `exists_split` extracts the first bad collider as a `Split` carrying a zero-bad prefix and a count bound. `ancestor_escape` shows the bad child’s leaked ancestry must reach a node in X or Y (its descendant cone misses Z , so it cannot dead-end). `bad_child_survives` and `escape_path_survives` prove that the rerouted escape chain stays inside the d -separation graph nodes. Reattaching the escape chain—backward into X or forward into Y —removes a moral jump of cost 1 and adds chains of cost 0; the strict decrease then follows by `countBadColliders_backwardEscape_append_suffix_lt` (resp. `countBadColliders_prefix_append_forwardEscape_lt`) and `omega`. As a deliberate design choice, the witness endpoints may change: rerouting is allowed to land on a *different* node of X or Y , which is what makes the escape construction total.

6.3 From Zero-Bad StaticRoute to OpenTrace to ActiveRoute

A static route with zero bad colliders compiles directly to an `OpenTrace`, because every junction is locally open:

Listing 10. Zero-bad compiler

```
theorem openTrace_of_countBadColliders_zero
  (hzero : countBadColliders arrival route = 0) :
  Nonempty (\Sigma finalDir, OpenTrace G Z (x, arrival) (y, finalDir))
```

The `OpenTrace` is then converted to a `BayesBallPath` and wrapped as an `ActiveRoute`. Finally, `ActiveRoute.to_activeTrail` yields the existential witness:

Listing 11. Final decompiler theorem

```
theorem activeWitness_of_not_dSeparated
  (hnot : \neg DAG.dSeparated G X Y Z) : ActiveWitness G X Y Z
```

The result is a constructive Σ -type containing the actual nodes $x \in X$, $y \in Y$, the final direction, and the active route.

6.4 The Closed Equivalence

Composing the optimizer (Section 5) and the decompiler yields the full equivalence on the well-typed domain—the bisimulation statement: on pairwise-disjoint X, Y, Z , the operational and static languages agree exactly.

Listing 12. Full d-separation equivalence (no proof debt)

```
theorem dSeparated_iff_dSeparates
  (hXYZ : DisjointSets X Y Z) :
  DAG.dSeparated G X Y Z <=> dSeparates G X Y Z
```

REMARK 1 (VERIFICATION STATUS). *The full Lean development typechecks (lake build). The graph-semantic core—both directions of `dSeparated_iff_dSeparates` and the entire reverse synthesis stack—contains no **sorry**. The only **sorry**s in the package are two intentional scaffold stubs in `InfoTheoryBridge.lean` (Section 8), which are out of scope for the equivalence claims.*

7 The PL Dictionary: Reframing Causal Graphs

To situate this work for a POPL audience, we reframe traditional causal-graph vocabulary in programming-languages terms:

Causal Inference	PL Framing
DAG	Dataflow AST / Dependency Graph Syntax
<code>DisjointSets X Y Z</code>	Affine ownership constraint (no aliasing of Z)
Active Trail	Operational trace of semantics
MAGWalk	Reachability abstract IR (compressed moral-graph walk)
Collider Jump (<code>MAGWalk.jump</code>)	Peephole optimization / semantic fold
<code>Trail</code> $\rightarrow$ <code>BayesBallPath</code>	Typestate tracking
<code>BayesBallPath</code> $\rightarrow$ <code>MAGWalk</code>	Certified trace optimizer
<code>StaticRoute</code> $\rightarrow$ <code>OpenTrace</code> $\rightarrow$ <code>ActiveRoute</code>	Semantic-flow decompiler / exploitability-certificate generator
Cut-Set / Mutual Information	Quantitative Information Flow (QIF) capacity bounds

Table 1. Reframing causal-graph concepts for a POPL audience.

8 Related Work

Our mechanization sits at the intersection of causal inference, mechanized probabilistic semantics, and verified compilation. We briefly survey the most closely related lines of work.

Mechanized Causal Inference and D-Separation. Formalizing the graph-theoretic foundations of causal inference is an active area of research. Recently, Chancelier, De Lara, and Heymann (2024) [2] mechanized d-separation as a binary relation using Coq and MathComp/SSReflect, providing a relational-algebraic treatment of graphical separation. Parallel to our work, Anna Zhang (2025) [9] formalized the semantics of conditional independence and d-separation in Coq, bridging structural properties with function-based definitions for experimental design.

Both developments adopt standard mathematical formulations without exposing the implicit textbook domain caveats. Our work contributes the first formalized counterexample and repair (the endpoint caveat) for the gap between syntactic trails and moralized node-deletion. We also shift from purely relational verification to a constructive bisimulation that generates executable IRs (e.g., OpenTrace), optimized in Lean 4 for computational witness synthesis.

Categorical and Topological D-Separation. Fritz and Klingler (2023) [3], extending work by Fritz and Liang (2023) [4], abstracted d-separation using string diagrams in Markov categories. Their framework bypasses moralization, reading conditional independence directly from topological connectedness in a generalized measure-theoretic space. Our approach is complementary: Markov categories provide a general topological theory, while our artifact targets the legacy operational graph definitions (moral graphs and trace trails) implemented in existing software. By defining DisjointSets as an affine ownership type, we show that classical d-separation behaves like a resource-restricted syntax within a broader categorical structure.

Verified Trace Bisimulation and Witness Extraction. The forward pipeline (soundness) translates unbounded trace semantics into a compressed step-relation (MAGWalk), analogous to the semantics-preserving translations in Leroy’s CompCert (2009) [6]. The reverse pipeline (completeness) deterministically unwinds reachability into a certified active trail, resembling Necula’s Proof-Carrying Code (1997) [7]: instead of returning a boolean assertion of an information leak, our decompiler synthesizes an explicit, machine-checkable exploitability certificate. This provides a graph-theoretic substrate for future verified static analyzers of quantitative information flow.

8.1 Quantitative Information-Flow Bridge & Technical Debts

Certified Shannon bounds. Security analysis has moved from non-interference (“0 or 1”) to measuring leakage in bits. The verified bisimulation here supplies the graph-semantic front end for such bounds: a closed d-separation equivalence licenses replacing a conditional-independence premise by a checked graph property in a cut-set capacity argument. The full finite chain is

$$\text{d-separation} \Rightarrow \text{conditional independence} \Rightarrow \text{conditional DPI} \Rightarrow \text{cut-set bound}.$$

This arXiv artifact does not import the quantitative project. Instead, InfoTheoryBridge.lean states the first bridge as an explicit local scaffold: the conditional-independence predicate ConditionalIndependence and the bridge theorem dSeparation_implies_conditional_independence are stubbed alongside placeholder structures FinitePMF and MarkovCompatible. This scaffold carries exactly two intentional [sorries](#), and it restates results proved in the separate anonymized artifact [1].

Engineering note: the graph artifact and the quantitative artifact are two physically separate Lake projects with near-isomorphic DAG definitions and no shared imports yet. The two [sorries](#) in this repository’s InfoTheoryBridge.lean file are a port placeholder for the proved semantic bridge in the cited anonymous artifact.

Remaining Technical Debts. This section separates proved theorem chains from remaining work in the broader pipeline:

- (1) **Debt 1: Structural Verma–Pearl derivation.** The companion artifact proves the semantic bridge `factorizes_dsep_implies_cond_indep` under a stated `FactorizesOverDAG` hypothesis. What remains open is the structural derivation of `FactorizesOverDAG` from a standard DAG factorization theorem; it is a narrow algebraic gap, not a gap in the semantic bridge theorem. (Sandbox implementation in `scratch_markov.lean`; pending integration.)
- (2) **Debt 2: General capacity metatheorem (dual KL witness).** The companion artifact proves concrete capacity certificates, including `capacity_le_of_kkt`, and uses them in cut-set bounds. A general Blahut–Arimoto/KKT concavity sufficiency theorem with a dual KL witness remains future work. (Sandbox implementation in `scratch_dual.lean`; pending integration.) The Gaussian-KL steps are carried as explicit external premises (`AssumesFanoBound`, `AssumesMissedCellBound`).
- (3) **Surface calculus and certified decision procedure.** The current artifact verifies the intrinsic DAG core. The planned first-order surface syntax, rank checker, elaboration soundness theorem, query conservativity theorem, and certified d-separation decision procedure are not yet mechanized.

9 Conclusion

A machine-checked counterexample shows that the two standard textbook characterizations of d-separation disagree when an endpoint lies in the conditioning set. The minimal repair—`DisjointSets X Y Z` as an affine ownership constraint—recovers full equivalence. We discharge both directions: a certified trace optimizer (soundness) and a constructive, witness-producing decompiler with proved bad-collider normalization (completeness), composing into the closed theorem `dSeparated_iff_dSeparates` with no proof debt in the graph-semantic core.

The verified bisimulation grounds a certified Shannon leakage-bound pipeline via a companion artifact. Two named technical debts in the quantitative pipeline are detailed in Section 8.

References

- [1] Anonymous. 2026. Certified Quantitative Information-Flow Bounds: A Finite Mechanized Cut-Set Pipeline. Companion artifact, under double-blind review; author and venue withheld for anonymity.
- [2] Jean-Philippe Chancelier, Michel De Lara, and Benjamin Heymann. 2024. Conditional Separation as a Binary Relation: A Coq Assisted Proof. Preprint. arXiv:2404.01777. Extended version available at hal-03315809.
- [3] Tobias Fritz and Andreas Klingler. 2023. The d-Separation Criterion in Categorical Probability. *Journal of Machine Learning Research (JMLR)* 24, 46 (2023), 1–49.
- [4] Tobias Fritz and Wendong Liang. 2023. Free gs-Monoidal Categories and Free Markov Categories. *Applied Categorical Structures* 31, 2 (2023), 21.
- [5] Steffen L. Lauritzen. 1996. *Graphical Models*. Oxford University Press, Oxford, UK.
- [6] Xavier Leroy. 2009. Formal verification of a realistic compiler. *Commun. ACM* 52, 7 (2009), 107–115.
- [7] George C. Necula. 1997. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, New York, NY, USA, 106–119.
- [8] Judea Pearl. 2009. *Causality* (2nd ed.). Cambridge University Press, Cambridge, UK.
- [9] Anna Zhang. 2025. *Formalizing Causal Models Through the Semantics of Conditional Independence*. Master’s thesis. Massachusetts Institute of Technology. Supervised by Adam Chlipala.